

Guía completa de árboles, HGB, XGB y MLP

División de regiones, no linealidad, ganancia, errores, métricas e hiperparámetros

De reglas por regiones a modelos de boosting y comparación uno a uno con una red MLP entrenada por backpropagation.

Índice

1. Resumen ejecutivo	3
2. Problema supervisado: notación base	3
2.1. Regresión vs clasificación	3
3. Qué es un árbol de decisión	4
3.1. Ruta, nodo y hoja	4
4. División del espacio en regiones	4
4.1. Idea geométrica	4
4.2. Función por partes	5
5. Cómo captura no linealidades	5
6. Hojas en clasificación y regresión	5
6.1. Hojas en clasificación	6
6.2. Hojas en regresión	6
6.3. Hojas en boosting	6
7. Ganancia: qué es y por qué importa	6
7.1. Ganancia general	7
7.2. Por qué es importante	7
8. Impureza en clasificación	7
8.1. Gini	7
8.2. Entropía	8
8.3. Ejemplo de ganancia con Gini	8
9. Ganancia en regresión	8
10. De un árbol a boosting	9
10.1. Idea de corrección sucesiva	9
11. Gradient Boosting matemáticamente	9
11.1. Caso de error cuadrático	10
12. XGBoost: función objetivo y ganancia	10
12.1. Expansión de segundo orden	10
12.2. Ganancia de división en XGBoost	11
13. Histogramas y bins	11
13.1. Problema de probar todos los cortes	11
13.2. Solución: agrupar en bins	11
13.3. Relación con HGB y XGB	11
14. HGB: HistGradientBoosting	12
14.1. Modelo aditivo	12
14.2. Parámetros principales de HGB	12
15. XGB: XGBoost	12
15.1. Parámetros principales de XGB	12
16. Comparación HGB vs XGB	13

17.Funciones de pérdida y errores	13
17.1. Error cuadrático medio: MSE	14
17.2. Raíz del error cuadrático medio: RMSE	14
17.3. Error absoluto medio: MAE	14
17.4. Coeficiente de determinación: R^2	14
17.5. Binary cross-entropy / log-loss	14
17.6. Softmax cross-entropy	14
18.Métricas de clasificación	15
18.1. Matriz de confusión binaria	15
18.2. Accuracy	15
18.3. Precision	15
18.4. Recall	15
18.5. F1-score	15
18.6. AUC-ROC	15
19.Comparación con MLP y backpropagation	15
19.1. MLP: estructura básica	15
19.2. Backpropagation	16
19.3. Mini-batch gradient descent	16
19.4. Nodo vs neurona	16
19.5. Hoja vs neurona de salida	16
19.6. Boosting vs backpropagation	17
20.No linealidad: árbol vs MLP	17
20.1. Árbol	17
20.2. MLP	17
21.Hiperparámetros organizados por función	17
21.1. Control de aprendizaje	17
21.2. Control de complejidad del árbol	18
21.3. Regularización	18
21.4. Velocidad por histogramas	18
22.Código base comentado	18
22.1. HGB para regresión	18
22.2. XGB para regresión	18
22.3. MLP para clasificación binaria	19
23.Cómo explicar todo en una exposición	19
23.1. Ruta recomendada	19
23.2. Frase de cierre	20
24.Checklist de conceptos esenciales	20
25.Ajuste práctico de hiperparámetros	20
25.1. Regla general	20
25.2. Señales de subajuste y sobreajuste	20
25.3. Qué mover si el modelo subajusta	20
25.4. Qué mover si el modelo sobreajusta	20
25.5. Orden recomendado de ajuste	21
26.Random Forest vs Boosting	21
27.Referencias breves	21

1 Resumen ejecutivo

Idea central

Un árbol de decisión no aprende una ecuación suave como una regresión lineal o una red neuronal. Aprende a **dividir el espacio de entrada en regiones**. Cada región termina en una hoja, y cada hoja guarda una predicción: una clase, una probabilidad, un valor continuo o una corrección al score del modelo.

Para entender HGB y XGB hay que unir cuatro ideas:

1. **Árbol de decisión**: divide el espacio con preguntas del tipo $x_j < c$.
2. **Ganancia**: mide qué tan buena es una división. Es la brújula que decide por dónde cortar.
3. **Boosting**: suma muchos árboles pequeños, cada uno corrigiendo errores del conjunto anterior.
4. **Histogramas / bins**: aceleran la búsqueda de cortes agrupando valores continuos en intervalos.

La comparación global es:

Aspecto	Árbol / HGB / XGB	MLP con backpropagation
Unidad básica	Nodo de decisión y hoja	Neurona
Operación	Preguntas $x_j < c$	$z = Wx + b$, activación $a = \sigma(z)$
No linealidad	Regiones por partes	Activaciones no lineales
Aprendizaje	Elegir divisiones y valores de hojas	Ajustar pesos y sesgos
Criterio	Ganancia, reducción de error o impureza	Gradiente de una función de pérdida
Modelo final	Suma de árboles o reglas	Composición de capas

2 Problema supervisado: notación base

Tenemos un conjunto de datos:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^p.$$

Cada x_i es un vector de características. En un ejemplo financiero del oro:

$$x_i = [\text{dólar}, \text{tasa}, \text{inflación}, \text{volatilidad}, \text{media móvil}, \dots].$$

El objetivo es aprender una función:

$$\hat{y} = f(x),$$

que prediga el valor real y .

2.1 Regresión vs clasificación

Tipo	Salida	Ejemplo
Regresión	Número continuo	Precio del oro mañana, retorno porcentual
Clasificación binaria	Clase 0 o 1	0 = baja, 1 = sube
Clasificación multiclase	Una de K clases	baja, estable, sube

Para clasificación binaria se puede modelar una probabilidad:

$$P(y = 1 \mid x) = p(x), \quad 0 \leq p(x) \leq 1.$$

La clase final se obtiene por umbral:

$$\hat{y} = \begin{cases} 1, & p(x) \geq 0.5, \\ 0, & p(x) < 0.5. \end{cases}$$

3 Qué es un árbol de decisión

Un árbol de decisión es un modelo que hace preguntas secuenciales sobre las variables de entrada. Cada pregunta divide los datos en dos ramas.

Ejemplo de preguntas

- ¿dólar < 3900?
- ¿volatilidad < 0.02?
- ¿tasa de interés < 0.08?
- ¿media móvil de 7 días > media móvil de 30 días?

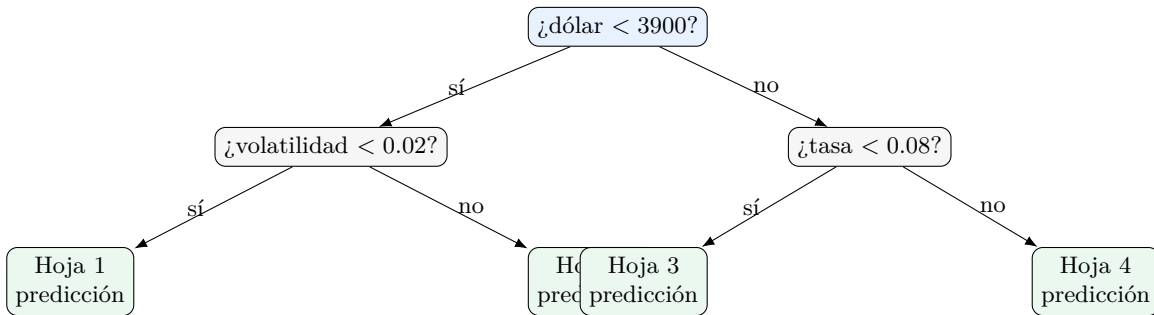
Matemáticamente, un nodo prueba una condición:

$$x_j < c,$$

con x_j una variable y c un punto de corte.

3.1 Ruta, nodo y hoja

- **Nodo:** lugar donde se hace una pregunta.
- **Rama:** resultado de la pregunta, por ejemplo izquierda si $x_j < c$ y derecha si $x_j \geq c$.
- **Hoja:** región final donde el árbol entrega una predicción.



4 División del espacio en regiones

4.1 Idea geométrica

Si hay dos variables, por ejemplo:

$$x_1 = \text{dólar}, \quad x_2 = \text{volatilidad},$$

cada observación es un punto en un plano. Una pregunta como:

$$x_1 < 3900$$

divide el plano en dos regiones:

$$R_L = \{x : x_1 < 3900\}, \quad R_R = \{x : x_1 \geq 3900\}.$$

Si después dentro de una región se hace otra pregunta:

$$x_2 < 0.02,$$

se crean más regiones. Al final, cada hoja representa una región:

$$R_m = \{x : \text{condición 1 y condición 2 y ...}\}.$$

4.2 Función por partes

Un árbol de regresión puede escribirse como:

$$f(x) = \sum_{m=1}^M c_m \mathbf{1}(x \in R_m),$$

con:

- R_m : región final u hoja.
- c_m : predicción de esa hoja.
- $\mathbf{1}(x \in R_m)$: función indicadora.

La indicadora vale:

$$\mathbf{1}(x \in R_m) = \begin{cases} 1, & x \in R_m, \\ 0, & x \notin R_m. \end{cases}$$

Por eso un árbol es una **función por partes**. No intenta usar una sola ecuación global. Usa diferentes predicciones en diferentes regiones.

5 Cómo captura no linealidades

Una relación lineal tiene forma:

$$\hat{y} = w_1 x_1 + w_2 x_2 + b.$$

Su frontera de decisión es una línea o hiperplano. Pero un árbol genera fronteras escalonadas por cortes sucesivos. Esto permite representar interacciones no lineales.

Ejemplo tipo XOR

Suponga:

Dólar	Volatilidad	Clase
bajo	baja	baja
bajo	alta	sube
alto	baja	sube
alto	alta	baja

Una línea simple no separa bien este patrón. Un árbol sí:

```
Si dolar < 3900:
  Si volatilidad < 0.02: baja
  Si volatilidad >= 0.02: sube
Si dolar >= 3900:
  Si volatilidad < 0.02: sube
  Si volatilidad >= 0.02: baja
```

Conclusión

El árbol captura no linealidad porque permite que el efecto de una variable dependa de otra. La volatilidad puede significar una cosa cuando el dólar es bajo y otra distinta cuando el dólar es alto.

6 Hojas en clasificación y regresión

6.1 Hojas en clasificación

En clasificación, una hoja no es literalmente una clase. La hoja es una región que guarda una distribución de clases o un score.

Ejemplo:

Hoja A: 30 datos de clase 0, 10 datos de clase 1.

Entonces:

$$P(y = 0 \mid \text{Hoja A}) = \frac{30}{40} = 0.75,$$

$$P(y = 1 \mid \text{Hoja A}) = \frac{10}{40} = 0.25.$$

La clase predicha es la clase mayoritaria:

$$\hat{y} = 0.$$

Precisión importante

La hoja no es la clase. La hoja es una región final del árbol. Dentro de esa región, el árbol asigna una clase, una probabilidad o una corrección al score.

6.2 Hojas en regresión

En regresión, una hoja suele guardar un valor continuo. Si en una hoja caen valores:

$$y = [2020, 2030, 2040],$$

la predicción puede ser el promedio:

$$\hat{y} = \frac{2020 + 2030 + 2040}{3} = 2030.$$

6.3 Hojas en boosting

En HGB y XGB, cada árbol no necesariamente entrega la clase final. Cada árbol aporta una corrección. Para clasificación binaria, el conjunto de árboles puede producir un score:

$$z = f_1(x) + f_2(x) + \dots + f_M(x).$$

Luego se convierte en probabilidad mediante sigmoide:

$$p = P(y = 1 \mid x) = \frac{1}{1 + e^{-z}}.$$

La clase final es:

$$\hat{y} = \begin{cases} 1, & p \geq 0.5, \\ 0, & p < 0.5. \end{cases}$$

7 Ganancia: qué es y por qué importa

Definición

La ganancia mide cuánto mejora una división. Si una pregunta separa mejor los datos, reduce la impureza o el error, y por tanto tiene alta ganancia.

En cada nodo, el árbol prueba muchas divisiones posibles:

$$x_j < c.$$

Para cada división calcula una ganancia y escoge la más alta.

7.1 Ganancia general

La forma general es:

$$\text{Ganancia} = \text{calidad antes de dividir} - \text{calidad despues de dividir.}$$

Si se mide impureza:

$$Gain = I(\text{padre}) - \left[\frac{n_L}{n} I(L) + \frac{n_R}{n} I(R) \right].$$

Donde:

- $I(\text{padre})$: impureza del nodo padre.
- $I(L)$: impureza del hijo izquierdo.
- $I(R)$: impureza del hijo derecho.
- n_L, n_R : muestras en izquierda y derecha.
- $n = n_L + n_R$.

7.2 Por qué es importante

La ganancia es importante porque decide la estructura del árbol:

1. El árbol prueba cortes posibles.
2. Calcula la ganancia de cada corte.
3. Escoge el corte con mayor ganancia.
4. Repite el proceso en cada nodo hijo.

Sin ganancia, el árbol dividiría a ciegas.

8 Impureza en clasificación

8.1 Gini

La impureza de Gini se define como:

$$Gini = 1 - \sum_{k=1}^K p_k^2.$$

Para dos clases:

$$Gini = 1 - p_0^2 - p_1^2.$$

Casos importantes:

Distribución	Cálculo	Gini
50/50	$1 - 0.5^2 - 0.5^2$	0.5
90/10	$1 - 0.9^2 - 0.1^2$	0.18
100/0	$1 - 1^2 - 0^2$	0

Gini bajo significa nodo más puro.

8.2 Entropía

Otra medida es la entropía:

$$H = - \sum_{k=1}^K p_k \log_2(p_k).$$

Para dos clases 50/50:

$$H = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1.$$

Para un nodo puro, por ejemplo $p_0 = 1, p_1 = 0$:

$$H = 0.$$

8.3 Ejemplo de ganancia con Gini

Nodo padre:

50 baja, 50 sube.

Entonces:

$$Gini_{padre} = 0.5.$$

Una división candidata produce:

L : 45 baja, 5 sube,

R : 5 baja, 45 sube.

Para la izquierda:

$$Gini_L = 1 - 0.9^2 - 0.1^2 = 0.18.$$

Para la derecha:

$$Gini_R = 1 - 0.1^2 - 0.9^2 = 0.18.$$

Como ambos hijos tienen 50 muestras:

$$Gini_{después} = \frac{50}{100}(0.18) + \frac{50}{100}(0.18) = 0.18.$$

La ganancia es:

$$Gain = 0.5 - 0.18 = 0.32.$$

Es una buena división.

9 Ganancia en regresión

En regresión no buscamos pureza de clases, sino reducción del error. Un nodo predice normalmente el promedio:

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

El error cuadrático en el nodo es:

$$SSE = \sum_{i=1}^n (y_i - \bar{y})^2.$$

Si dividimos en izquierda y derecha:

$$Gain = SSE_{padre} - (SSE_L + SSE_R).$$

Ejemplo

Datos en un nodo:

$$y = [2000, 2010, 2020, 2100, 2110, 2120].$$

Sin dividir, el promedio es 2060 y el error es grande. Si se divide en:

$$L = [2000, 2010, 2020], \quad R = [2100, 2110, 2120],$$

los promedios son 2010 y 2110, y el error cae mucho. Esa división tiene alta ganancia.

10 De un árbol a boosting

Un árbol individual puede ser limitado. Boosting crea un conjunto de árboles:

$$F_M(x) = f_0(x) + \eta f_1(x) + \eta f_2(x) + \dots + \eta f_M(x).$$

Donde:

- $F_M(x)$: modelo final después de M árboles.
- $f_m(x)$: árbol número m .
- η : learning rate.

10.1 Idea de corrección sucesiva

El modelo empieza con una predicción inicial $F_0(x)$. Luego cada árbol intenta corregir los errores del modelo actual.

$$F_m(x) = F_{m-1}(x) + \eta f_m(x).$$

Lectura intuitiva

El árbol 1 captura un patrón grueso. El árbol 2 corrige errores del árbol 1. El árbol 3 corrige errores restantes. Muchos árboles pequeños forman un modelo flexible.

11 Gradient Boosting matemáticamente

Gradient boosting generaliza la idea de residuos. Dada una pérdida:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, F(x_i)),$$

el siguiente árbol se ajusta a la dirección de descenso del error:

$$r_i^{(m)} = - \left[\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}.$$

Estos $r_i^{(m)}$ son pseudo-residuos. El nuevo árbol intenta aproximarlos:

$$f_m(x_i) \approx r_i^{(m)}.$$

Finalmente:

$$F_m(x) = F_{m-1}(x) + \eta f_m(x).$$

11.1 Caso de error cuadrático

Si:

$$l(y, F) = \frac{1}{2}(y - F)^2,$$

entonces:

$$\frac{\partial l}{\partial F} = F - y.$$

La dirección negativa es:

$$-(F - y) = y - F.$$

Es decir, el nuevo árbol aprende el residuo:

$$r_i = y_i - \hat{y}_i.$$

12 XGBoost: función objetivo y ganancia

XGBoost optimiza una función objetivo regularizada:

$$Obj = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^M \Omega(f_k).$$

La regularización del árbol suele escribirse como:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2.$$

Donde:

- T : número de hojas.
- w_j : valor de la hoja j .
- γ : costo por crear hojas/divisiones.
- λ : regularización L2 de los valores de hoja.

12.1 Expansión de segundo orden

XGBoost usa gradiente y Hessiano:

$$g_i = \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i},$$

$$h_i = \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}.$$

Para una hoja j se suman:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i.$$

El peso óptimo de la hoja es:

$$w_j^* = -\frac{G_j}{H_j + \lambda}.$$

La calidad de una hoja se relaciona con:

$$-\frac{1}{2} \frac{G_j^2}{H_j + \lambda} + \gamma.$$

12.2 Ganancia de división en XGBoost

Si un nodo se divide en izquierda y derecha:

$$G = G_L + G_R, \quad H = H_L + H_R,$$

la ganancia es:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma.$$

Interpretación

La división se acepta si el beneficio de crear dos hojas supera el beneficio de quedarse con una sola hoja, descontando el costo γ .

13 Histogramas y bins

13.1 Problema de probar todos los cortes

Si una variable continua tiene muchos valores, un árbol exacto podría probar miles de cortes:

$$x < c_1, \quad x < c_2, \quad x < c_3, \dots$$

Esto es costoso.

13.2 Solución: agrupar en bins

Un bin es un intervalo. En vez de usar todos los valores reales, se discretiza:

$$x \rightarrow b(x).$$

Ejemplo:

Valor del dólar	Bin
3800	1
3820	1
3850	2
3900	3
3950	4

Entonces se prueban cortes como:

$$b(x) \leq 1, \quad b(x) \leq 2, \quad b(x) \leq 3.$$

Árbol rápido por bins

Un árbol por bins acelera el entrenamiento porque reduce el número de cortes candidatos. Sacrifica precisión fina en el punto de corte, pero gana mucha velocidad.

13.3 Relación con HGB y XGB

- HGB usa histogramas por diseño.
- XGB puede usar histogramas con `tree_method="hist"`.

14 HGB: HistGradientBoosting

HGB suele referirse a `HistGradientBoostingRegressor` o `HistGradientBoostingClassifier` de scikit-learn. Es una versión de gradient boosting basada en histogramas, pensada para ser rápida en datos medianos o grandes.

14.1 Modelo aditivo

HGB construye:

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^M f_m(x).$$

La diferencia práctica es que los cortes se buscan mediante bins/histogramas.

14.2 Parámetros principales de HGB

Parámetro	Qué controla	Interpretación
<code>loss</code>	Función de pérdida	Define qué error minimiza el modelo: cuadrático, absoluto, log-loss, etc.
<code>learning_rate</code>	Peso de cada árbol	Valor η en $F_m = F_{m-1} + \eta f_m$. Bajo: más lento y estable. Alto: más rápido y riesgoso.
<code>max_iter</code>	Número de árboles	Cantidad máxima de etapas de boosting. Similar a <code>n_estimators</code> en XGB.
<code>max_leaf_nodes</code>	Máximo de hojas	Controla cuántas regiones puede crear cada árbol. Más hojas: más flexibilidad.
<code>max_depth</code>	Profundidad máxima	Limita cuántas preguntas encadenadas puede hacer un árbol.
<code>min_samples_leaf</code>	Muestras mínimas por hoja	Evita hojas con muy pocos datos. Reduce sobreajuste.
<code>l2_regularization</code>	Penalización L2	Suaviza valores extremos de las hojas.
<code>max_bins</code>	Número máximo de bins	Controla la discretización de variables continuas. Más bins: más precisión. Menos bins: más velocidad.
<code>early_stopping</code>	Parada temprana	Detiene entrenamiento si no mejora la validación.
<code>validation_fraction</code>	Fracción de validación	Parte del entrenamiento usada para monitorear early stopping.
<code>random_state</code>	Semilla	Hace reproducibles resultados aleatorios.

15 XGB: XGBoost

XGB suele referirse a XGBoost, una implementación muy optimizada de gradient boosting con árboles. Se caracteriza por:

- uso de gradiente y Hessiano,
- regularización explícita,
- manejo eficiente de datos dispersos,
- métodos exactos, aproximados e histogramas,
- soporte CPU/GPU según configuración.

15.1 Parámetros principales de XGB

Parámetro	Qué controla	Interpretación
<code>objective</code>	Función objetivo	Define la tarea: <code>reg:squarederror</code> , <code>binary:logistic</code> , <code>multi:softprob</code> , etc.
<code>n_estimators</code>	Número de árboles	Cantidad de árboles del ensemble.
<code>learning_rate / eta</code>	Peso de cada árbol	Controla la magnitud de cada corrección.
<code>max_depth</code>	Profundidad máxima	Árboles más profundos capturan más interacciones, pero sobreajustan más.
<code>min_child_weight</code>	Evidencia mínima en hijo	Mínima suma de Hessianos para permitir una hoja/hijo. Más alto: más conservador.
<code>subsample</code>	Filas por árbol	Proporción de muestras usadas por árbol. Reduce sobreajuste.
<code>colsample_bytree</code>	Columnas por árbol	Proporción de variables usadas por árbol. Aumenta diversidad.
<code>gamma</code>	Costo de división	Ganancia mínima necesaria para dividir. Más alto: menos divisiones.
<code>reg_alpha</code>	Regularización L1	Favorece modelos más escasos.
<code>reg_lambda</code>	Regularización L2	Suaviza pesos de hojas y reduce sobreajuste.
<code>tree_method</code>	Método de construcción	<code>hist</code> usa histogramas/bins; <code>exact</code> prueba cortes exactos si está disponible.
<code>max_bin</code>	Bins en histograma	Número máximo de bins cuando se usa <code>hist</code> .
<code>eval_metric</code>	Métrica de monitoreo	Por ejemplo <code>rmse</code> , <code>mae</code> , <code>logloss</code> , <code>auc</code> .
<code>early_stopping_rounds</code>	Parada temprana	Detiene entrenamiento si no mejora durante ciertas rondas.
<code>random_state</code>	Semilla	Reproducibilidad.
<code>n_jobs</code>	Paralelización CPU	Número de núcleos usados.

16 Comparación HGB vs XGB

Aspecto	HGB	XGB
Librería común	scikit-learn	xgboost
Idea dominante	Boosting rápido por histogramas	Boosting avanzado con regularización y segundo orden
Velocidad	Muy rápido en datos medianos/grandes	Muy rápido y muy optimizado
Configuración	Más simple	Más flexible y extensa
Regularización	L2 y controles de árbol	L1, L2, gamma, subsampling, colsampling
Bins	<code>max_bins</code> por diseño	<code>tree_method="hist"</code> , <code>max_bin</code>
Curva de aprendizaje	Más fácil	Más compleja
Competencias	Menos dominante	Muy usado en Kaggle y datos tabulares

Resumen

HGB es excelente cuando se quiere un modelo potente y sencillo dentro de scikit-learn. XGB es mejor cuando se quiere control fino, regularización fuerte y máximo rendimiento mediante ajuste de hiperparámetros.

17 Funciones de pérdida y errores

17.1 Error cuadrático medio: MSE

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Penaliza mucho los errores grandes. Es suave y fácil de optimizar.

17.2 Raíz del error cuadrático medio: RMSE

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

Está en las mismas unidades de y . Si y es precio del oro en dólares, RMSE también está en dólares.

17.3 Error absoluto medio: MAE

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

Es más robusto a valores atípicos que MSE/RMSE.

17.4 Coeficiente de determinación: R^2

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Interpretación:

- $R^2 = 1$: predicción perfecta.
- $R^2 = 0$: predice igual que usar el promedio.
- $R^2 < 0$: peor que usar el promedio.

17.5 Binary cross-entropy / log-loss

Para clasificación binaria:

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)].$$

Donde $p = P(y = 1 \mid x)$.

Si $y = 1$ y p es bajo, la pérdida es grande. Si $y = 0$ y p es alto, también es grande.

17.6 Softmax cross-entropy

Para multiclase:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

La pérdida es:

$$L = - \sum_{k=1}^K y_k \log(p_k),$$

con y_k codificado one-hot.

18 Métricas de clasificación

18.1 Matriz de confusión binaria

	Predicho 1	Predicho 0
Real 1	TP	FN
Real 0	FP	TN

- TP: verdaderos positivos.
- TN: verdaderos negativos.
- FP: falsos positivos.
- FN: falsos negativos.

18.2 Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

18.3 Precision

$$Precision = \frac{TP}{TP + FP}.$$

Mide cuántas predicciones positivas fueron correctas.

18.4 Recall

$$Recall = \frac{TP}{TP + FN}.$$

Mide cuántos positivos reales fueron detectados.

18.5 F1-score

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}.$$

Equilibra precision y recall.

18.6 AUC-ROC

Mide la capacidad del modelo para ordenar positivos por encima de negativos al variar el umbral de clasificación.

19 Comparación con MLP y backpropagation

19.1 MLP: estructura básica

Una MLP transforma la entrada mediante capas:

$$h_1 = \sigma(W_1 x + b_1),$$

$$h_2 = \sigma(W_2 h_1 + b_2),$$

$$\hat{y} = g(W_3 h_2 + b_3).$$

Donde:

- W_l : matriz de pesos de la capa l .
- b_l : sesgo.
- σ : activación no lineal, por ejemplo ReLU o sigmoide.
- g : función de salida, por ejemplo sigmoide, softmax o identidad.

19.2 Backpropagation

Backpropagation calcula gradientes de la pérdida respecto a cada peso:

$$\frac{\partial L}{\partial W_l}.$$

Después actualiza:

$$W_l \leftarrow W_l - \eta \frac{\partial L}{\partial W_l}.$$

19.3 Mini-batch gradient descent

Con un batch B de tamaño m :

$$L_B = \frac{1}{m} \sum_{i \in B} L(y_i, \hat{y}_i).$$

La actualización usa el gradiente promedio del batch:

$$W_l \leftarrow W_l - \eta \frac{\partial L_B}{\partial W_l}.$$

19.4 Nodo vs neurona

Concepto	Árbol	MLP
Unidad	Nodo	Neurona
Operación	$x_j < c$	$z = Wx + b$
Salida local	Rama izquierda/derecha	Activación $\sigma(z)$
Tipo de decisión	Dura/discreta	Suave/continua
Aprende	Variable y corte	Pesos y sesgo

19.5 Hoja vs neurona de salida

- Una hoja se alcanza por una ruta de decisiones.
- Una neurona de salida combina activaciones de la capa anterior.

En boosting:

$$z = \sum_{m=1}^M f_m(x).$$

En una MLP:

$$z = W_{out}h + b_{out}.$$

Ambos pueden pasar por sigmoide:

$$p = \frac{1}{1 + e^{-z}}.$$

Pero el z se construye de manera diferente.

19.6 Boosting vs backpropagation

Aspecto	Boosting	Backpropagation
Cómo corrige errores	Agrega árboles nuevos	Ajusta pesos existentes
Estructura	Crece con más árboles	Arquitectura fija en capas
Parámetros aprendidos	Cortes y hojas	Pesos y sesgos
Dirección de mejora	Pseudo-residuos / gradientes sobre predicción	Gradientes sobre pesos
Learning rate	Peso de cada árbol	Tamaño del paso en pesos

20 No linealidad: árbol vs MLP

20.1 Árbol

El árbol usa una función por partes:

$$f(x) = \sum_{m=1}^M c_m \mathbf{1}(x \in R_m).$$

La no linealidad viene de las regiones R_m .

20.2 MLP

La MLP usa composición de funciones no lineales:

$$f(x) = W_3 \sigma(W_2 \sigma(W_1 x + b_1) + b_2) + b_3.$$

La no linealidad viene de σ .

Comparación conceptual

El árbol construye no linealidad con cortes. La MLP construye no linealidad con activaciones. HGB/XGB superponen muchos árboles; una MLP compone muchas capas.

21 Hiperparámetros organizados por función

21.1 Control de aprendizaje

Idea	HGB	XGB
Número de árboles	<code>max_iter</code>	<code>n_estimators</code>
Peso de cada árbol	<code>learning_rate</code>	<code>learning_rate / eta</code>
Parada temprana	<code>early_stopping</code>	<code>early_stopping_rounds</code>

Regla práctica:

learning rate bajo + más árboles $\Rightarrow$ aprendizaje más gradual.

21.2 Control de complejidad del árbol

Idea	HGB	XGB
Profundidad	max_depth	max_depth
Número de hojas	max_leaf_nodes	Indirecto con profundidad y gamma
Mínimo por hoja	min_samples_leaf	min_child_weight
Costo de dividir	No tan explícito	gamma

21.3 Regularización

Idea	HGB	XGB
L2	l2_regularization	reg_lambda
L1	No central	reg_alpha
Muestreo de filas	No típico en HGB clásico	subsample
Muestreo de columnas	Limitado según versión	colsample_bytree

21.4 Velocidad por histogramas

Idea	HGB	XGB
Método histograma	Por diseño	tree_method="hist"
Número de bins	max_bins	max_bin

22 Código base comentado

22.1 HGB para regresión

```
from sklearn.ensemble import HistGradientBoostingRegressor

hgb = HistGradientBoostingRegressor(
    loss="squared_error", # perdida para regresion
    learning_rate=0.05, # peso de cada arbol
    max_iter=300, # numero maximo de arboles
    max_leaf_nodes=31, # maximo de hojas por arbol
    max_depth=None, # profundidad maxima, None = sin limite explicito
    min_samples_leaf=20, # minimo de muestras por hoja
    l2_regularization=0.1, # penalizacion L2
    max_bins=255, # bins para variables continuas
    random_state=42 # reproducibilidad
)

hgb.fit(X_train, y_train)
y_pred = hgb.predict(X_test)
```

22.2 XGB para regresión

```
from xgboost import XGBRegressor

xgb = XGBRegressor(
    objective="reg:squarederror", # objetivo de regresion
    n_estimators=500, # numero de arboles
    learning_rate=0.03, # peso de cada arbol
    max_depth=4, # profundidad maxima
```

```

min_child_weight=5, # evidencia minima para crear hijos
subsample=0.8, # porcentaje de filas por arbol
colsample_bytree=0.8, # porcentaje de columnas por arbol
gamma=0.1, # ganancia minima para dividir
reg_alpha=0.0, # regularizacion L1
reg_lambda=1.0, # regularizacion L2
tree_method="hist", # metodo por histogramas
max_bin=256, # bins del metodo hist
eval_metric="rmse", # metrica de evaluacion
random_state=42,
n_jobs=-1
)

xgb.fit(X_train, y_train)
y_pred = xgb.predict(X_test)

```

22.3 MLP para clasificación binaria

```

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(p, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 1),
    nn.Sigmoid()
)

criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for X_batch, y_batch in loader:
    p_hat = model(X_batch)
    loss = criterion(p_hat, y_batch)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

23 Cómo explicar todo en una exposición

23.1 Ruta recomendada

1. Empezar con el problema: predecir y desde variables x .
2. Explicar árbol simple: nodos, ramas, hojas.
3. Mostrar que cada hoja es una región del espacio.
4. Explicar no linealidad por regiones.
5. Introducir ganancia: criterio para elegir cortes.
6. Explicar clasificación: Gini, entropía, probabilidades.
7. Explicar regresión: reducción de error cuadrático.
8. Pasar a boosting: muchos árboles corrigen errores.
9. Explicar HGB: acelerar con bins/histogramas.
10. Explicar XGB: gradiente, Hessiano y regularización.
11. Comparar con MLP: pesos, activaciones y backpropagation.
12. Cerrar con hiperparámetros y métricas.

23.2 Frase de cierre

Síntesis

Un árbol aprende dividiendo el espacio en regiones; la ganancia decide qué cortes hacen mejores regiones; HGB acelera esa construcción usando bins; XGB refina la decisión con gradientes, Hessianos y regularización; y una MLP resuelve el mismo problema de otra forma, ajustando pesos continuos mediante backpropagation.

24 Checklist de conceptos esenciales

- Un nodo hace una pregunta $x_j < c$.
- Una hoja es una región final, no una clase en sí misma.
- En clasificación, la hoja guarda probabilidades, conteos o scores.
- En regresión, la hoja guarda un valor continuo o una corrección.
- La ganancia mide reducción de impureza o error.
- La no linealidad del árbol viene de particiones por regiones.
- Boosting suma árboles para corregir errores sucesivos.
- HGB usa histogramas/bins para velocidad.
- XGB usa gradiente, Hessiano y regularización explícita.
- Una MLP usa pesos, sesgos, activaciones y backpropagation.
- En boosting se corrige agregando árboles; en MLP se corrige actualizando pesos.

25 Ajuste práctico de hiperparámetros

25.1 Regla general

El ajuste de HGB y XGB se puede pensar como balance entre tres fuerzas:

capacidad *vs* regularización *vs* velocidad.

- Si el modelo subajusta, necesita más capacidad: más árboles, más profundidad o más hojas.
- Si el modelo sobreajusta, necesita más regularización: menos profundidad, más mínimos por hoja, mayor L2, mayor gamma, subsampling.
- Si el entrenamiento es lento, necesita menos bins, menos árboles o árboles más pequeños.

25.2 Señales de subajuste y sobreajuste

Situación	Error entrenamiento	Error validación
Subajuste	Alto	Alto
Buen ajuste	Bajo/medio	Bajo/medio
Sobreajuste	Muy bajo	Alto

25.3 Qué mover si el modelo subajusta

- Aumentar `max_iter` o `n_estimators`.
- Aumentar `max_depth` o `max_leaf_nodes`.
- Bajar demasiado `min_samples_leaf` o `min_child_weight` puede aumentar capacidad, pero con cuidado.
- Disminuir regularización si está demasiado alta.

25.4 Qué mover si el modelo sobreajusta

- Reducir `max_depth`.
- Reducir `max_leaf_nodes`.

- Aumentar `min_samples_leaf` o `min_child_weight`.
- Aumentar `reg_lambda`, `reg_alpha`, `l2_regularization` o `gamma`.
- Usar `subsample` y `colsample_bytree` en XGB.
- Usar early stopping.

25.5 Orden recomendado de ajuste

1. Definir una métrica correcta: RMSE, MAE, log-loss, F1, AUC, etc.
2. Separar entrenamiento, validación y prueba. En series de tiempo, respetar el orden temporal.
3. Empezar con parámetros conservadores.
4. Ajustar `learning_rate` y número de árboles juntos.
5. Ajustar profundidad/hojas.
6. Ajustar regularización.
7. Revisar métricas en validación, no solo en entrenamiento.

26 Random Forest vs Boosting

Aunque esta guía se centra en HGB y XGB, conviene ubicar Random Forest.

Aspecto	Random Forest	HGB / XGB
Idea	Muchos árboles independientes	Árboles secuenciales que corrigen errores
Entrenamiento	Paralelo	Secuencial
Predicción	Promedio o voto	Suma de correcciones
Error que ataca	Reduce varianza	Reduce sesgo y error residual
Riesgo	Menos sensible a tuning	Puede sobreajustar si se ajusta mal
Interpretación	Ensemble de votos	Trayectoria de correcciones

Diferencia clave

Random Forest promedia muchos árboles fuertes entrenados con aleatoriedad. Boosting suma muchos árboles, normalmente más pequeños, donde cada nuevo árbol intenta corregir lo que el ensemble anterior todavía hace mal.

27 Referencias breves

- Scikit-learn: documentación oficial de `HistGradientBoostingRegressor` y `HistGradientBoostingClassifier`. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html>
- Scikit-learn: documentación oficial de `GradientBoostingRegressor`. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html>
- XGBoost: documentación oficial de parámetros. <https://xgboost.readthedocs.io/en/stable/parameter.html>
- Chen, T. y Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. <https://arxiv.org/abs/1603.02754>